



Практически реализуем модель кодом

Практическая реализация модели — ниже. Она:

- корректирует веса при усилении кризиса;
- нормирует веса так, чтобы их сумма была равна 1;
- рассчитывает ожидаемый риск $p \cdot L$;
- применяет жёсткие ограничения безопасности;
- учитывает стоимость, время, усилие и неопределённость;
- сравнивает обычный, умеренно кризисный и тяжёлый кризисный режимы;
- исключает вариант, не прошедший ограничения, даже если его выгода высока.

```
from dataclasses import dataclass
from typing import Dict, List

CRITERIA = ("benefit", "safety", "cost", "time", "effort")

@dataclass(frozen=True)
class Option:
    name: str
    benefit: float          # 0..10, больше — лучше
    injury_prob: float      # 0..1
    injury_severity: float  # 0..10
    cost: float             # 0..10, меньше — лучше
    time: float             # 0..10, меньше — лучше
    effort: float           # 0..10, меньше — лучше
    confidence: float       # 0..1
    reversible: bool
    safety_validated: bool

    @property
    def risk(self) -> float:
        """
        Нормированный ожидаемый ущерб:
        вероятность травмы × тяжесть последствий.
        """
        return min(
            self.injury_prob * self.injury_severity / 10.0,
            1.0
        )
```

```

@dataclass(frozen=True)
class Constraints:
    max_risk: float = 0.10
    min_confidence: float = 0.60
    require_reversible: bool = False

def validate_weights(weights: Dict[str, float]) -> None:
    if set(weights) != set(CRITERIA):
        raise ValueError(
            f"Ожидаемые критерии: {CRITERIA}"
        )

    if any(value < 0 for value in weights.values()):
        raise ValueError("Веса не могут быть отрицательными")

    total = sum(weights.values())

    if abs(total - 1.0) > 1e-9:
        raise ValueError(
            f"Сумма весов должна быть равна 1, получено {total}"
        )

def adjust_weights(
    base_weights: Dict[str, float],
    crisis_multipliers: Dict[str, float]
) -> Dict[str, float]:
    """
    Усиливает или ослабляет критерии кризисными множителями
    и затем нормирует сумму весов к 1.
    """
    validate_weights(base_weights)

    raw_weights = {
        criterion: base_weights[criterion]
        * crisis_multipliers.get(criterion, 1.0)
        for criterion in CRITERIA
    }

    total = sum(raw_weights.values())

    if total <= 0:
        raise ValueError("Сумма скорректированных весов должна быть положительной")

    return {
        criterion: raw_weights[criterion] / total
        for criterion in CRITERIA
    }

def interpolate_weights(
    normal_weights: Dict[str, float],
    crisis_weights: Dict[str, float],
    crisis_severity: float
) -> Dict[str, float]:

```

```

"""
Плавный переход:
severity=0 – обычный режим;
severity=1 – полный кризис.
"""

if not 0 <= crisis_severity <= 1:
    raise ValueError("Тяжесть кризиса должна быть в диапазоне [0, 1]")

validate_weights(normal_weights)
validate_weights(crisis_weights)

mixed = {
    criterion:
        (1 - crisis_severity) * normal_weights[criterion]
        + crisis_severity * crisis_weights[criterion]
    for criterion in CRITERIA
}

total = sum(mixed.values())

return {
    criterion: mixed[criterion] / total
    for criterion in CRITERIA
}

def passes_constraints(
    option: Option,
    constraints: Constraints
) -> bool:
    """
    Жёсткие ограничения применяются до оптимизации.
    Небезопасный вариант не может компенсировать риск выгодой.
    """
    return (
        option.safety_validated
        and option.risk <= constraints.max_risk
        and option.confidence >= constraints.min_confidence
        and (
            not constraints.require_reversible
            or option.reversible
        )
    )

def calculate_score(
    option: Option,
    weights: Dict[str, float],
    uncertainty_penalty: float = 0.20
) -> float:
    """
    Итоговая полезность:

    выгода
    - риск
    - стоимость
    """

```

```

- время
- усилие
- штраф за неопределённость
"""
validate_weights(weights)

normalized = {
    "benefit": option.benefit / 10.0,
    "safety": option.risk,
    "cost": option.cost / 10.0,
    "time": option.time / 10.0,
    "effort": option.effort / 10.0,
}

score = (
    weights["benefit"] * normalized["benefit"]
    - weights["safety"] * normalized["safety"]
    - weights["cost"] * normalized["cost"]
    - weights["time"] * normalized["time"]
    - weights["effort"] * normalized["effort"]
)

uncertainty_cost = uncertainty_penalty * (
    1.0 - option.confidence
)

return score - uncertainty_cost

def rank_options(
    options: List[Option],
    weights: Dict[str, float],
    constraints: Constraints
) -> List[dict]:
    results = []

    for option in options:
        allowed = passes_constraints(option, constraints)

        result = {
            "name": option.name,
            "allowed": allowed,
            "risk": option.risk,
            "confidence": option.confidence,
            "reason": "ok",
            "score": float("-inf"),
        }

        if allowed:
            result["score"] = calculate_score(
                option,
                weights
            )
        else:
            result["reason"] = (
                "нарушено ограничение безопасности, "

```

```

        "допустимого риска или уверенности"
    )

    results.append(result)

return sorted(
    results,
    key=lambda item: item["score"],
    reverse=True
)

def choose_option(
    options: List[Option],
    weights: Dict[str, float],
    constraints: Constraints
) -> dict:
    ranking = rank_options(
        options,
        weights,
        constraints
    )

    if not ranking or not ranking[0]["allowed"]:
        return {
            "name": "NO-GO: действие отложить",
            "ranking": ranking,
        }

    return {
        "name": ranking[0]["name"],
        "ranking": ranking,
    }

def print_report(title: str, result: dict) -> None:
    print(f"\n{title}")
    print("-" * len(title))
    print(f"Рекомендация: {result['name']}")

    for row in result["ranking"]:
        if row["score"] == float("-inf"):
            score = "-inf"
        else:
            score = f"{row['score']:.3f}"

        print(
            f"{row['name']}: "
            f"допустим={row['allowed']}, "
            f"риск={row['risk']:.3f}, "
            f"уверенность={row['confidence']:.2f}, "
            f"оценка={score}, "
            f"причина={row['reason']}"
        )

```

```

def main() -> None:
    options = [
        Option(
            name="Лезть по стволу ёлки",
            benefit=10,
            injury_prob=0.35,
            injury_severity=8,
            cost=1,
            time=5,
            effort=9,
            confidence=0.90,
            reversible=False,
            safety_validated=False,
        ),

        Option(
            name="Использовать сертифицированную лестницу",
            benefit=10,
            injury_prob=0.02,
            injury_severity=6,
            cost=5,
            time=4,
            effort=3,
            confidence=0.90,
            reversible=True,
            safety_validated=True,
        ),

        Option(
            name="Обратиться к профессионалу",
            benefit=9,
            injury_prob=0.01,
            injury_severity=5,
            cost=7,
            time=3,
            effort=1,
            confidence=0.95,
            reversible=True,
            safety_validated=True,
        ),

        Option(
            name="Купить мёд",
            benefit=8,
            injury_prob=0.0,
            injury_severity=0,
            cost=2,
            time=1,
            effort=0,
            confidence=0.99,
            reversible=True,
            safety_validated=True,
        ),
    ]

    normal_weights = {

```

```

        "benefit": 0.30,
        "safety": 0.25,
        "cost": 0.15,
        "time": 0.15,
        "effort": 0.15,
    }

    crisis_multipliers = {
        "benefit": 0.70,
        "safety": 2.50,
        "cost": 2.00,
        "time": 1.00,
        "effort": 0.50,
    }

    crisis_weights = adjust_weights(
        normal_weights,
        crisis_multipliers
    )

    scenarios = [
        (
            "Обычный режим",
            normal_weights,
            Constraints(
                max_risk=0.20,
                min_confidence=0.50,
                require_reversible=False,
            ),
        ),
        (
            "Умеренный кризис",
            interpolate_weights(
                normal_weights,
                crisis_weights,
                crisis_severity=0.50,
            ),
            Constraints(
                max_risk=0.10,
                min_confidence=0.70,
                require_reversible=True,
            ),
        ),
        (
            "Тяжёлый кризис",
            crisis_weights,
            Constraints(
                max_risk=0.05,
                min_confidence=0.85,
                require_reversible=True,
            ),
        ),
    ]

```

```

print("Веса по сценариям:")

for scenario_name, weights, _ in scenarios:
    readable_weights = {
        key: round(value, 3)
        for key, value in weights.items()
    }

    print(f"{scenario_name}: {readable_weights}")

for scenario_name, weights, constraints in scenarios:
    result = choose_option(
        options,
        weights,
        constraints
    )

    print_report(
        scenario_name,
        result
    )

if __name__ == "__main__":
    main()

```

Логика результата

Для варианта «лезть по стволу ёлки» задано:

```

injury_prob=0.35
injury_severity=8

```

Поэтому нормированный ожидаемый риск равен:

$$R = 0.35 \cdot \frac{8}{10} = 0.28.$$

В обычном режиме он может быть рассмотрен, но помечен как непроверенный и небезопасный. В кризисном режиме ограничение, например,

$$R \leq 0.10,$$

автоматически исключает этот вариант.

Таким образом, алгоритм не пытается «выиграть» рискованным вариантом за счёт высокой выгоды:

```

если вариант не проходит ограничения безопасности:
    вариант исключается
иначе:
    рассчитывается итоговая полезность

```


Для запуска сохраните код в файл `decision_model.py` и выполните:

```
python decision_model.py
```

Математический смысл итоговой стратегии:

\$\$

a^*

$\arg\max_{a \in A_{\text{safe}}}$

$\left[$

$\sum_i w_i u_i(a)$

$-\lambda(1-q(a))$

$\right],$

\$\$

где A_{safe} — множество вариантов, прошедших ограничения безопасности. Практический вывод для исходной задачи: прямое лазание на ёлку не является допустимым кандидатом; выбирается безопасная альтернатива — сертифицированное оборудование, специалист или покупка мёда.